

Разработка информационной системы автоматизированной деятельности коммерческого учреждения

Задание на модуль №1

I. Микросервис ввода и модификации данных о клиентах условного коммерческого учреждения (банка). Модуль «Клиенты».

Цель: Разработать учебную подсистему ввода и модификации данных о клиентах условного банка.

Среда и технологии: язык и фреймворк фиксированные: **Java + Spring**. Работу предлагается выполнить в ОС произвольной версии. Рекомендуемая база данных – h2 (для запуска из любого ПК) или любая другая реляционная, которая работает в памяти. При старте приложения база должна заполняться автоматически.

Задание:

1. Выбрать поля базы данных для ввода и хранения информации о клиентах для выполнения лабораторной работы согласно номеру варианта.

Наименование поля	Тип поля	Обязательное (да, нет)	Варианты
Фамилия	Текстовый	Да	Любой
Имя	Текстовый	Да	Любой
Отчество	Текстовый	Да	Любой
Дата рождения	Дата	Да	Любой
Пол	radio	Да	Четный
Серия паспорта	Текстовый	Да	Любой
№ паспорта	Текстовый (с маской)	Да	Любой
Кем выдан	Текстовый	Да	Любой
Дата выдачи	Дата	Да	Любой
Идент. номер	Текстовый (с маской)	Да	Любой
Место рождения	Текстовый	Да	Любой
Город факт. проживания	Список (от 5 городов)	Да	Любой
Адрес факт. проживания	Текстовый	Да	Любой
Телефон дом	Текстовый (с маской)	Нет	Любой
Телефон моб	Текстовый (с маской)	Нет	Любой

Е-mail	Текстовый	Нет	Любой
Место работы	Текстовый	Нет	Нечетный
Должность	Текстовый	Нет	Нечетный
Город прописки	Список (от 5 городов)	Да	1,4,7... ($N \% 3 == 1$)
Адрес прописки	Текстовый	Да	2,5,8... ($N \% 3 == 2$)
Семейное положение	Список	Да	Любой
Гражданство	Список	Да	Любой
Инвалидность	Список	Да	Любой
Пенсионер	Checkbox	Да	Любой
Ежемесячный доход	Денежный	Нет	Любой
Военнообязанный	Checkbox	Да	3,6,9... ($N \% 3 == 0$)

2. Разработать схему данных в которой для элементов типа *Список* предусмотрены отдельные справочники.

3. Реализовать схему данных в выбранной СУБД.

4. Разработать форму добавления, удаления и редактирования клиентов банка.

5. Разработать форму "Список клиентов", список отсортировать по фамилиям.

6. Создать **микросервис для REST-подключения** к базе данных.

7. На клиентской стороне нужно создать модуль "Работа с клиентами", который реализует две формы, аналогичные разработанным макетам, технологически это обязательно **Web-клиент** (в качестве фреймворка можно использовать любой JS-framework).

8. Предусмотреть проверку корректности введенных значений в каждом из полей с выдачей предупреждения в случае ошибки или незаполнения обязательного поля.

ВАЛИДАЦИЯ ПРОВОДИТСЯ ДВАЖДЫ – НА КЛИЕНТЕ и в REST

9. Добавить в базу данных через разработанную программу не менее 5 клиентов, включая себя.

10. На хорошую оценку нужно выполнить тестирование разработанного ПО на WebDriver + любые_нужные_mock + etc. **База данных в памяти H2.** Требования к тестам приведены ниже:

Автоматизированное тестирование проводится по тем же пунктам, по которым будет проводиться защита работы:

Примечание: под данными Иванов Иван Иванович понимаются данные студента, который защищает лабораторную работу.

1. Введите данные клиента Иванов Иван Иванович ...(все поля) и сохраните его в базе.

Введите те же данные еще раз (должно быть сообщение об ошибке) - **зачтено**.

Если ошибки нет - проверьте таблицу клиентов прямо в базе (два и более одинаковых клиента - **незачтено**).

2. Введите данные клиента Иванов Иван Иванович ...(все остальные поля).

Введите другого клиента с тем же паспортом (должно быть сообщение об ошибке) - **зачтено**.

Если ошибки нет - проверьте таблицу клиентов прямо в базе (два и более одинаковых паспорта - **незачтено**).

3. Введите данные клиента Иванов Иван Иванович ...(все остальные поля).

Введите другого клиента с тем же Идент. номером (должно быть сообщение об ошибке) - **зачтено**.

Если ошибки нет - проверьте таблицу клиентов прямо в базе (два и более одинаковых Идент. номера - **незачтено**).

4. Введите данные клиента 1234 Иван Иванович ...(все остальные поля). Должно быть сообщение об ошибке. Проверьте таблицу клиентов прямо в базе. Если клиента с фамилией 1234 там нет - **зачтено**. Иначе **незачтено**.

5. Введите данные клиента Иванов [пробел вместо имени] Иванович ...(все остальные поля).

Должно быть сообщение об ошибке. Проверьте таблицу клиентов прямо в базе. Если клиента с фамилией Иванов там нет - **зачтено**. Иначе **незачтено**.

6. Введите данные клиента Иванов Иван Иванович ...(все поля). В поле **дата рождения** укажите текст, не являющийся датой (в усложненном варианте: 31 февраля 2015 года, в наиболее сложном варианте: 29 февраля 2015 года).

Должно быть сообщение об ошибке. Проверьте таблицу клиентов прямо в базе. Если клиента с фамилией Иванов там нет - **зачтено**. Иначе **незачтено**.

..... аналогично нужно проверить все **поля с маской**.

7. Введите данные клиента Иванов Иван Иванович ... (все обязательные поля, кроме одного обязательного поля, например, Дата рождения). Должно быть сообщение об ошибке - **зачтено**. Если программа пишет в базу - **незачтено**.

..... аналогично нужно проверить все **обязательные поля**.

8. Введите данные клиента Иванов Иван Иванович ... (ТОЛЬКО все обязательные поля). Должна быть выполнена запись клиента - **зачтено**. Если программа ожидает одно или несколько необязательных полей - **незачтено**.

9. Вы разработали приложение типа клиент-сервер или web-приложение, это решение близкое к реальным системам управления банком. Поэтому для каждого поля с маской, а также для обязательных полей, должна выполняться **двойная проверка маски и обязательности поля, т.е. отдельно на клиенте и на сервере**. Найдите в своей программе и **выделите в отчете полужирным шрифтом два фрагмента**, которые эту проверку выполняют. Один фрагмент первичной проверки в коде клиента или html-странички, а другой – фрагмент валидации в сервере.

Оценки.

4-6. Сервис компилируется и запускается в браузере.

7-8. Есть модульные и/или интеграционные тесты.

9-10. Есть функциональные тесты.

На проверку:

1. Отчет оформляется в формате **PDF** (обязательно) и содержит **краткое** описание разработанного ПО с кодом.

2. Внимание! **Полный набор кодовой базы сдается в предложенный репозиторий в виде Pull Request (в ветку со своей фамилией).**